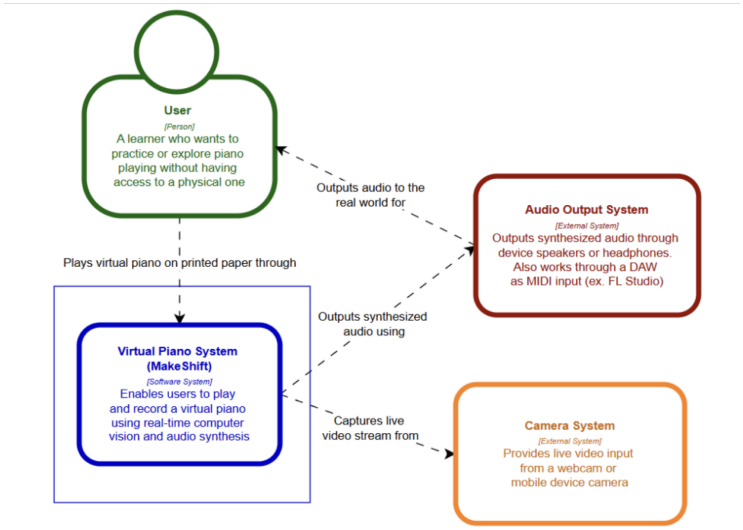
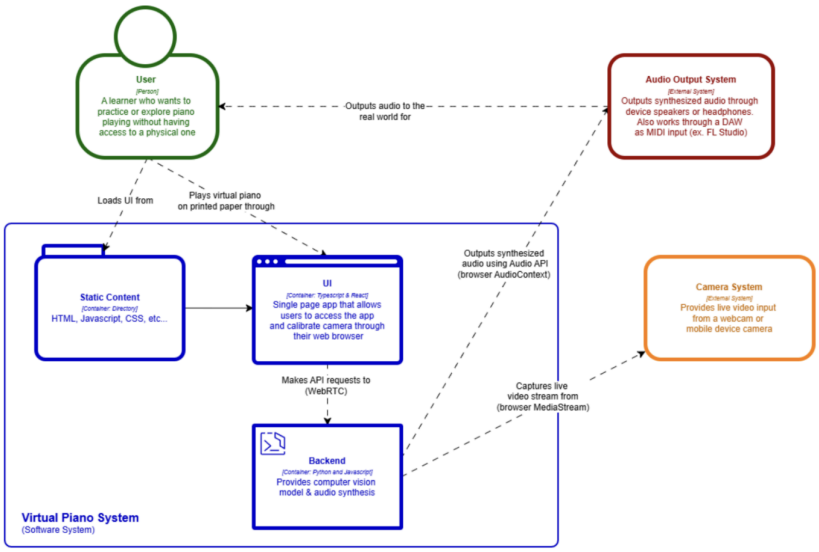
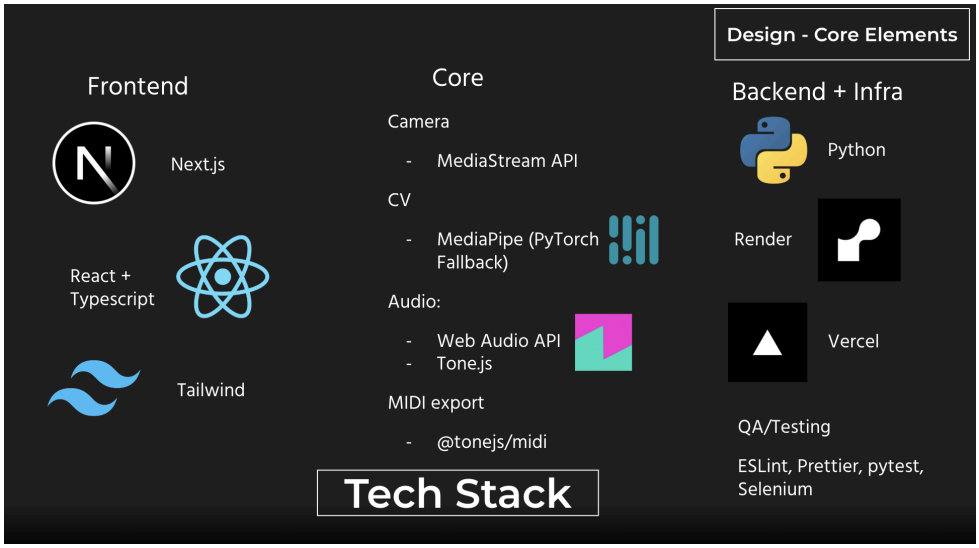
Design Diagram







Figma

Figma Link

Design Trade

Backend:

- BD-1: Render
    - Persistent container processes that stay alive between requests, with flat-rate billing. Free tier sleeps after inactivity.
- Vercel
    - Frontend optimized serverless platform.
- Railway
    - Nearly identical to render but uses usage based billing

Criterion	Weight	BD-1 (Render)	BD-2 (Vercel)	BD-3 (Railway)
Process Persistence	30	9 (*.3=2.7)	2 (*.3=.6)	9 (*.3=2.7)
API Response Latency	25	8 (2)	9 (2.25)	8 (2)
Scalability	20	8 (1.6)	3 (.6)	7 (1.4)
Github CI/CD Integration	15	8 (1.2)	9 (1.35)	8 (1.2)
Documentation & Developer Experience	10	8 (.8)	9 (.9)	7 (.7)
Total score	100	8.3	5.7	8

Render was chosen because its flat-rate pricing is more predictable than railway's usage-based model. Vercel got eliminated early because its serverless architecture can't maintain a persistent webrtc connection. The two deciding criteria really were process persistence and API response latency, where Render and Railway tied but Render's billing model pretty much broke the tie.

MIDI Subsystem:

- MIDI-1: Synchronized Event Queue
    - The necessary data is packaged by the CV subsystem and then pushed into a FIFO queue protected by a mutex lock. Then the MIDI subsystem pops the information and sends it off.
- MIDI-2: Lock-Free Single Producer Single Consumer (SPSC) Ring Buffer

    - This approach eliminates the use of a mutex, lowering the overall overhead of the MIDI subsystem. The SPSC ring buffer allows the MIDI subsystem to read from a buffer whenever its read index doesn't match the CV subsystem's write index.
- MIDI-3: Framerate Efficiency
    - This approach moves the calculations of MIDI generation over to the MIDI subsystem rather than having the CV subsystem do the math. The coordinate data would be written to a buffer and then read by the MIDI subsystem to handle the calculation.

Designs	Latency and Jitter (35%)	Thread Overhead (20%)	Note Accuracy (25%)	CPU Utilization (10%)	Resolution of Velocity (10%)	Total (after weights)
MIDI-1	3 = 1.05	2 = 0.4	7 = 1.75	4 = 0.4	3 = 0.3	3.9
MIDI-2	9 = 3.15	9 = 1.8	7 = 1.75	8 = 0.8	3 = 0.3	7.8
MIDI-3	7 = 2.45	8 = 1.6	8 = 2	3 = 0.3	9 = 0.9	7.25

The efficiency in latency of the MIDI-2 design is what makes it stand out from the rest. By utilizing a lock-free SPSC ring buffer, the locking of resources from traditional use of mutex locks is mitigated. This means that both threads are going at the same time. While this leads to increased complexity in implementation because of race conditions, the outcome is that the MIDI subsystem is lightweight and allows more resources to be used for the CV subsystem. While the third alternative design allowed for much higher resolution of velocity, the increased amount of computation due to the number of frames being analyzed increases the risk of higher latency and randomness in latency.

<u>Computer Vision</u>

- <u>ID:</u> DA-1
- <u>Title:</u> Key tracking
- <u>Description:</u> OpenCV is capable of tracking the angle of a surface. For this design alternative, you would track the surface of each physical key to determine if it was pressed instead of the position of the finger. This would also require a change to the piano sheet since a sheet of paper would not have key depth.

- <u>ID:</u> DA-2
- <u>Title:</u> Front-back camera placement
- <u>Description:</u> The current design uses a single camera in front of the user at an angle. Alternatively, the camera could be placed directly in front of the hands.

- <u>ID:</u> DA-3
- <u>Title:</u> PyTorch

- <u>Description:</u> The current design uses the MediaPipe library. One alternative is to train our own model using PyTorch.

Weighted

Alternative ID	Accuracy Score	Accessibility Score	Speed Score	Flexibility Score	Simplicity Score	Total Score
DA-1	8 = 2.4	4 = 0.8	7 = 1.4	7 = 1.05	7 = 1.05	6.7
DA-2	8 = 2.4	8 = 1.6	7 = 1.4	7 = 1.05	7 = 1.05	7.5
DA-3	9 = 2.7	8 = 1.6	8 = 1.6	6 = 0.9	3 = 0.45	7.25

The final choice is DA-2 which is the front-back camera placement. To be clear this is the choice among the alternatives because the assignment does not ask me to compare it to the original subsystem. It appears to be the best simply because it's the most balanced. The accuracy score evens out because it sacrifices accuracy in one dimension for improved accuracy in another, but it's hard to say if this is how it would work in reality without any testing. Both other alternatives have a significant compromise, one in accessibility, and the other in simplicity. Assuming that MediaPipe can accurately determine depth from the front facing angle, which it appears to be able to do to an extent based on my "Learn a skill" assignment, this is by far the most promising alternative. The other drawbacks with seeing the piano sheet and the setup could theoretically have relatively simple workarounds. For example, the edges of the piano sheet could be folded upwards to help the application understand the boundaries of the keyboard, and a laptop could either be placed behind the surface of the piano sheet at a lower elevation, or the piano sheet itself could be placed on an elevated surface.

<u>Audio Engine</u>

1. Alternative 1
    a. AUDIO-1: Front-Load Frontend
    b. If no audio notes are loaded or a new configuration by the user has been ordered, send to the backend to create and send the new audio notes needed. The frontend then stores all the audio notes and applies any audio manipulation (for example, ADSR or volume) and outputs to the user.
2. Alternative 2
    a. AUDIO-2: Frontend Audio Engine
    b. The frontend will create, store, and load from its own memory all the audio notes and output for the user. The frontend will also be in charge of any audio manipulation.
3. Alternative 3
    a. AUDIO-3: Dummy Frontend

b. The frontend receives the final audio to be played from the backend and outputs it. The frontend performs no audio manipulations or stores; the backend does all the work and sends to the frontend the final audio file.

Alternative IDs	Low Latency (40%)	Bandwidth Usage (20%)	Portability (10%)	Sound Production (15%)	User Burden (15%)	Total	Rank
AUDIO-1	9 = 3.6	8 = 1.6	6 = .6	5 = .75	6 = .90	7.45	1
AUDIO-2	9 = 3.6	10 = 2	2 = .2	8 = 1.2	1 = .15	7.15	2
AUDIO-3	3 = 1.2	2 = .4	10 = .1	5 =.75	10 = 1.5	4.85	3

AUDIO-1 utilizes both the backend and the frontend. For most of the piano process, it acts the same as AUDIO-2 by utilizing the keys cached in the frontend. However, AUDIO-1 does not have to create the sound waves on the user's device but places the load on our own computers, which contrasts AUDIO-2. AUDIO-3 overall has a bad record due to the lack of speed. However, AUDIO-3 works best for portability and better experience for the device of the user as everything gets run in the backend. AUDIO-1 does the best in using both the backend and the frontend to its advantage.